

chinesecolorRGB0,128,0

Pigs: Protocol-Native Phased Orchestration for LLM API Requests

Anonymous

Abstract

Large Language Model (LLM) agents typically interact with providers through stateless API calls, lacking built-in mechanisms for planning, execution verification, and failure recovery. While frameworks like ReAct and Reflexion add multi-stage reasoning within a single conversation, they rely on the LLM itself to manage control flow—an approach that is fragile, protocol-specific, and opaque to downstream tooling. We present **Pigs**, a phased orchestration middleware that wraps LLM API requests with a deterministic Pre→Executor→Post state machine using lightweight text control markers (PIGEND/PIGFAIL) for routing. Pigs preserves the complete protocol-native HTTP request—method, path, headers, and JSON body—across three API protocols (OpenAI Chat Completions, Anthropic Messages, OpenAI Responses), modifying only the model identifier, current user input, and phase transcript. Phase subrequests re-enter the local proxy via HTTP loopback, transparently inheriting channel selection, model mapping, body cleaning, and retry logic. External tool execution is supported through a bounded in-memory continuation store with TTL and capacity limits. We describe the architecture, key design decisions, and implementation in a 13-crate Rust workspace.

1 Introduction

Modern LLM-based agents interact with model providers through HTTP APIs that are fundamentally stateless: a client sends messages and receives a completion, with no built-in notion of planning, execution verification, or failure recovery. To achieve complex multi-step behavior, practitioners have developed orchestration frameworks—from simple chain-of-thought prompting (?) to interleaved reasoning-acting loops (?) and reflective self-improvement (??). These approaches share a common limitation: they place the LLM in charge of its own control flow. The model must decide when to plan, when to execute, and when to reflect, using natural language cues that are inherently ambiguous and error-prone.

This design creates three problems. First, **control-flow fragility**: if the model deviates from the expected reasoning pattern (e.g., skipping the planning phase or prematurely declaring completion), the entire agent loop breaks. Second, **protocol coupling**: most orchestration frameworks are tightly coupled to a single API protocol (typically OpenAI Chat Completions), requiring reimplementations when switching providers. Third, **opacity**: external tooling (proxies, observability platforms, cost monitors) cannot inspect or influence the orchestration logic because it is embedded inside the agent’s prompt engineering rather than exposed as a system-level concern.

We argue that LLM orchestration should be elevated from a prompting technique to a *middleware concern*—a transparent layer between the client and the provider that enforces a deterministic control-flow structure while leaving semantic decisions to the model. This separation mirrors the classical middleware pattern in distributed systems, where transport-level concerns (routing, retry, load balancing) are handled outside the application logic.

We present **Pigs**, a phased orchestration middleware that implements this vision. Pigs wraps each LLM API request with a three-phase state machine:

1. **Pre** (planning): the model analyzes the task, identifies required information, and formulates an execution plan. If the task is trivially simple, the model may short-circuit with a PIGEND marker.
2. **Executor** (execution): the model executes the plan, gathering information and producing output. Tool calls are returned to the external agent for execution.
3. **Post** (review): the model independently reviews the executor’s output against the goal. It may confirm completion (PIGEND), signal failure (PIGFAIL), or continue working (no marker).

The state machine is driven by *control markers*—short text tokens (PIGEND, PIGFAIL) that the model appends to its output. These markers are detected by the middleware, not the model’s own reasoning; they serve as routing signals, not as content. Crucially, the middleware strips these markers from the visible output, so the client sees only the model’s substantive text.

Pigs makes four key contributions:

1. **Phased orchestration via text control markers**: a deterministic Pre→Executor→Post state machine that uses lightweight text markers for routing, without injecting orchestration-specific tools or system prompts into the model’s context.
2. **Protocol-native request preservation**: the complete HTTP request—method, path/query, headers, and JSON body—is preserved across three API protocols, with only the model identifier, current user input, and phase transcript modified. Unknown fields, media blocks, and provider extensions pass through untouched.
3. **HTTP loopback transport**: phase subrequests re-enter the local proxy via HTTP, transparently inheriting channel selection, model mapping, body cleaning, thinking-effort injection, and retry logic—without requiring a separate in-process dispatch path.
4. **Bounded in-memory continuation**: external tool execution is supported through a continuation store with TTL, capacity limits, and tombstone-based error reporting, never persisting authentication headers to disk.

The remainder of this paper describes the architecture (??), implementation (??), related work (??), and design discussion (??).

2 Background and Motivation

2.1 LLM API Protocols

Three API protocols dominate the LLM provider landscape:

- **OpenAI Chat Completions** (`/chat/completions`): messages with roles (system, user, assistant, tool), tool calls, and streaming via Server-Sent Events (SSE) terminated by `[DONE]`.
- **Anthropic Messages** (`/v1/messages`): content blocks (text, tool_use, tool_result, thinking), system as a top-level field, and SSE terminated by `message_stop`.
- **OpenAI Responses** (`/responses`): a unified input/output array with typed items (message, reasoning, function_call, function_call_output), instructions as a top-level field, and SSE terminated by `response.completed`.

Each protocol has distinct message structure, tool-call representation, streaming format, and termination semantics. A middleware that supports all three must either (a) convert between protocols (lossy) or (b) preserve each protocol’s native format (lossless). Pigs chooses the latter.

2.2 Multi-Stage LLM Orchestration

Several lines of work add structure to LLM interactions:

ReAct (?) interleaves reasoning traces and actions in a single LLM output. The model decides when to think and when to act, with no external control flow. This is elegant but fragile: the model may skip planning, act prematurely, or fail to reflect on errors.

Reflexion (?) adds a reflection loop where the model receives verbal feedback and stores it in episodic memory for the next attempt. While effective, the reflection trigger is model-driven—there is no guarantee the model will reflect at the right time.

Self-Refine (?) uses a single LLM as generator, critic, and refiner. The control flow is implicit in the prompt structure, and the model may deviate from the expected generate-critic-refine sequence.

Plan-and-Solve (?) explicitly separates planning from execution in a two-phase prompt. However, both phases occur within a single LLM call, and there is no external verification of whether the plan was followed.

These approaches share a common pattern: they embed control flow *inside* the prompt, relying on the model’s instruction-following ability to maintain phase discipline. Pigs inverts this: control flow is *external* to the model, enforced by a deterministic state machine, while the model contributes only semantic content and routing signals.

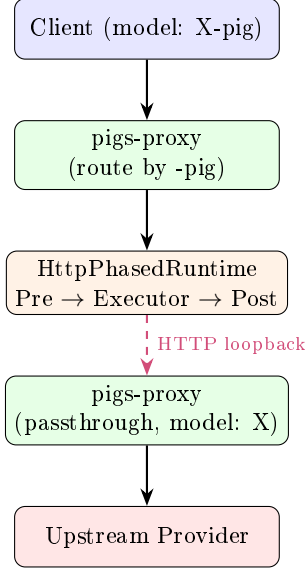


Figure 1: Request flow for a `-pig` model. Phase subrequests re-enter the proxy via HTTP loopback (dashed), inheriting passthrough features.

2.3 LLM Middleware

The concept of LLM middleware—a transparent layer between client and provider—has been discussed in position papers (?) but remains under-explored in terms of implemented systems. Existing proxy-like systems (e.g., LiteLLM, OpenRouter) focus on model routing and cost optimization, not on orchestration logic. TokenMizer (?) adds session state management as a proxy feature, but does not impose phase structure.

Pigs occupies a distinct niche: it is both a middleware (transparent, protocol-preserving, non-invasive) and an orchestrator (enforcing a phase structure with control markers). No prior system combines these two roles.

3 System Design

Pigs is structured as a three-layer system: a *proxy* layer that routes HTTP requests, a *phased runtime* layer that orchestrates the Pre→Executor→Post state machine, and a *CLI* layer that provides an interactive agent interface. This section describes the core design of each layer.

3.1 Architecture Overview

The system operates as a single-process server listening on one port (default 3927). All three API protocols are served from the same port, distinguished by request path:

- `/chat/completions` → OpenAI Chat
- `/v1/messages` → Anthropic Messages
- `/responses` → OpenAI Responses

Requests are routed by model identifier: models without the `-pig` suffix are passed through to the upstream provider (with retry, body cleaning, and thinking-effort injection); models with the `-pig` suffix enter the phased runtime. Each configured model is automatically exposed as two identifiers: `{model}` (passthrough) and `{model}-pig` (phased).

?? illustrates the request flow. When a `-pig` request arrives, the proxy constructs a `HttpRequestEnvelope` and hands it to the `HttpPhasedRuntime`. Each phase subrequest is sent back to the proxy via HTTP loopback (with an internal token header), re-entering the passthrough path with the `-pig` suffix removed. This design ensures that phase subrequests transparently inherit all proxy-level features.

Algorithm 1 Phase state machine transition

```
1: function ADVANCE(output)
2:   phase  $\leftarrow$  current phase
3:   if phase = Pre then
4:     if detect(output) = PIGEND then
5:       return Complete
6:     else if detect(output) = PIGFAIL then
7:       return replan(output)
8:     else
9:       pre_output  $\leftarrow$  strip(output)
10:      return Continue(Executor)
11:    end if
12:  else if phase = Executor then
13:    executor_outputs  $\leftarrow$  strip(output)
14:    return Continue(Post)
15:  else
16:    if detect(output) = PIGEND then
17:      return Complete
18:    else if detect(output) = PIGFAIL then
19:      return replan(output)
20:    else
21:      if post_iterations  $\geq$  max then
22:        return Error(PostBudgetExceeded)
23:      end if
24:      post_iterations += 1
25:      return Continue(Post)
26:    end if
27:  end if
28: end function
```

▷ Short-circuit: trivially simple task

▷ Failure: return to Pre

▷ Always proceed to Post
▷ *phase* = Post

▷ Stay in Post

3.2 Phase State Machine

The core of Pigs is a pure state machine with three phases and three transition signals. The state is encapsulated in `OrchestrationState`, which tracks the current phase, accumulated outputs, failure history, and iteration counters.

The state machine has several key properties:

Markerless Post stays in Post. When the Post phase produces output without a control marker, the state machine continues in Post (not back to Executor). This design choice prevents infinite Executor $\leftrightarrow$ Post loops: the model is expected to make incremental progress within Post until it either completes or fails.

Budget exhaustion is an error, not success. When the Post iteration budget is exhausted, the state machine returns an `OrchestrationError`, not a successful completion. This prevents the system from silently accepting incomplete work.

Failure preserves history. When PIGFAIL triggers a replan, the failure output is preserved and presented to the next Pre phase as numbered failure records (“1st failure: ...”, “2nd failure: ...”). This allows the model to learn from previous attempts.

?? illustrates the complete state machine. The Pre phase can short-circuit to Complete (trivially simple tasks), and the Post phase has a self-loop for markerless continuation.

3.3 Control Markers

Control markers are short text tokens that the model appends to its output to signal phase transitions:

- **PIGEND:** signals successful completion. The turn ends, and the accumulated visible text is returned to the client.
- **PIGFAIL:** signals that the current approach has failed and the task should be replanned from Pre. Marker detection follows two rules:

Last non-empty line only. Only the final non-empty line of the model’s output is checked for a marker. Earlier occurrences (e.g., the marker appearing in a sentence like “the system outputs PIGEND

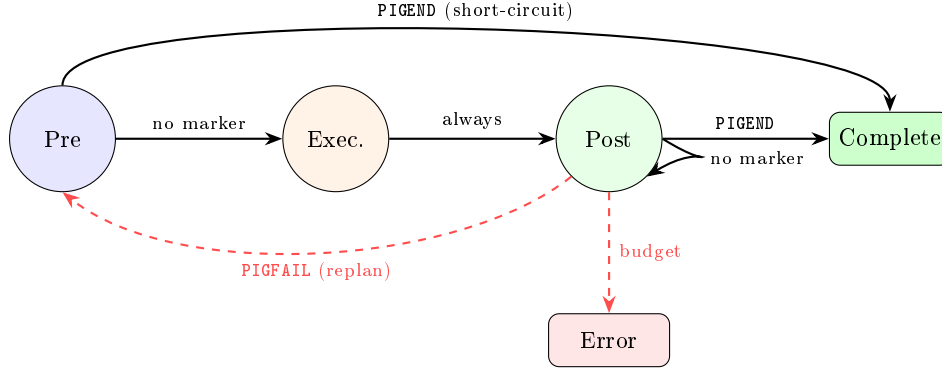


Figure 2: Phase state machine transitions. Solid arrows are forward transitions; dashed red arrows are failure and budget-exhaustion paths. The curved arrow from Post back to Pre represents PIGFAIL-triggered replanning. The self-loop on Post (right side) represents markerless continuation.

when done”) are ignored. This prevents false positives from prose mentions.

Reason required. A marker is valid only if at least one non-marker line precedes it. A bare PIGEND with no preceding content is rejected, forcing the model to provide a reason before signaling completion.

Markers are stripped from the visible output via `strip_markers`, which removes exact control lines while preserving prose that happens to contain marker words.

3.4 Protocol-Native Request Envelope

The `HttpRequestEnvelope` is the central data structure that preserves the complete protocol-native request:

```

pub struct HttpRequestEnvelope {
    pub method: String,           // HTTP method
    pub path_and_query: String,   // path + query string
    pub headers: Vec<HeaderPair>, // raw header bytes
    pub body: Value,              // complete JSON body
    pub protocol: Protocol,       // Chat / Anthropic / Responses
    pub client_model: String,     // e.g. "gpt-4-pig"
    pub real_model: String,       // e.g. "gpt-4" (-pig stripped)
    pub stream: bool,             // streaming requested?
    // ... internal fields for current user location,
    //      tool result IDs, tool result groups
}

```

Each phase clones the envelope and applies *point mutations*:

1. **Model:** replaced with the real (non--pig) model name.
2. **Current user input:** appended with the phase-specific prompt suffix (Pre questions, Executor plan, Post review instructions).
3. **Phase transcript:** native assistant/tool messages from prior phases are appended to the message history.
4. **Stream:** set to `true` for streaming phase subrequests.

All other fields—system/instructions, historical messages, tools/tool_choice, reasoning parameters, cache controls, metadata, media blocks, and unknown provider extensions—remain untouched. This ensures that provider-specific features (e.g., Anthropic’s `cache_control`, OpenAI’s `reasoning_effort`) pass through without modification.

3.5 HTTP Loopback Transport

Each phase subrequest is sent back to the local proxy via HTTP, not through an in-process function call. The `LoopbackPhaseTransport` implements the `PhaseTransport` trait:

- Converts wildcard listen addresses (`0.0.0.0, [:::]`) to loopback (`127.0.0.1, :::1`).
- Disables environment HTTP proxy settings (`.no_proxy()`) to prevent external proxy interception.

- Adds a random internal token header (`x-pigs-phase-loopback`) that the proxy validates to identify legitimate phase subrequests.
- Strips hop-by-hop headers and the internal token before forwarding to the upstream provider.

This design has two benefits. First, phase subrequests transparently inherit all proxy-level features: channel selection, model mapping, body cleaning (e.g., removing empty content messages), thinking-effort injection, and retry logic. Second, the control flow is explicit and debuggable—each phase subrequest appears in proxy logs as a normal HTTP request, and the internal token distinguishes it from client requests.

3.6 Bounded In-Memory Continuation

When the model returns tool calls during a phase, the runtime pauses and returns the tool calls to the client (or external agent). The complete phase state—including the orchestration state, phase transcript, visible text, and pending tool IDs—is stored in a `ContinuationStore`:

- **TTL**: each continuation expires after a configurable duration (default 30 minutes).
- **Capacity**: at most N continuations are retained (default 256); the oldest is evicted when capacity is reached.
- **Tombstones**: when a continuation is expired, evicted, or consumed, a tombstone is placed for each tool ID, enabling distinct error reporting (Expired vs. Evicted vs. Consumed vs. Unknown).
- **No auth headers**: authentication headers are cleared before storing; the continuation uses the incoming request’s headers on resume.

When the next request arrives with tool results, the runtime matches them by tool-call ID, validates protocol and model consistency, and resumes the paused phase. Partial parallel results (multiple tool calls returning in separate requests) are accumulated until all pending IDs are present.

3.7 Continuous SSE Output

For streaming requests, the runtime produces a continuous SSE stream that concatenates visible text from all phases. A *marker line buffer* holds back the last non-empty line of each phase’s output until the phase completes, then checks for control markers. If a marker is found, the held-back line is stripped; otherwise, it is forwarded. This prevents `PIGEND`/`PIGFAIL` from leaking into the client-visible stream, even when markers are split across network chunks.

The outer SSE encoder generates protocol-native event sequences for all three protocols:

- **Chat**: `chat.completion.chunk` deltas with `role/content/tool_calls`, terminated by `finish_reason` + `[DONE]`.
- **Anthropic**: `message_start` → `content_block_start/delta/stop` → `message_delta` → `message_stop`.
- **Responses**: `response.created` → `output_item/content_part/text_delta/done` → `response.completed`, with monotonic sequence numbers.

Error streams emit protocol-native error events (Chat: `error` in data; Anthropic: `event: error`; Responses: `response.failed`) without any success terminal frame.

4 Implementation

Pigs is implemented as a Rust workspace with 14 crates, organized in five layers:

Layer	Crates	Role
0	<code>pigs-core</code>	Core types: <code>Message</code> , <code>ContentBlock</code> , <code>ApiClient</code> trait
1	<code>pigs-config</code> , <code>pigs-permissions</code> , <code>pigs-session</code> , <code>pigs-prompts</code>	Config, permissions, session, prompts
2	<code>pigs-llm</code> , <code>pigs-tools</code> , <code>pigs-mcp</code>	LLM clients, built-in tools, MCP client
3	<code>pigs-api</code> , <code>pigs-proxy</code>	Phased runtime, HTTP proxy + loopback
4	<code>pigs-cli</code> , <code>pigs-tui</code> , <code>pigs</code>	Agent logic, terminal UI, product binary

4.1 Protocol Codec

The `ProtocolCodec` in `pigs-api` handles parsing, mutation, and response extraction for all three protocols. It validates the “current user input” (the last user message, not just the last non-system message) and identifies trailing tool-result groups for continuation matching.

Each protocol has dedicated parsing logic:

- **Chat:** messages array with role-based content; tool calls as `tool_calls` array; tool results as `role: "tool"` messages.
- **Anthropic:** messages with content block arrays; tool calls as `tool_use` blocks; tool results as `tool_result` blocks within user messages.
- **Responses:** input array with typed items; tool calls as `function_call` items; tool results as `function_call_output` items.

The codec clones the original body and modifies only the JSON path leading to the current user input, leaving all other fields structurally identical.

4.2 HTTP Runtime

The `HttpPhasedRuntime` drives the phase loop. For each phase, it:

1. Constructs a phase request via `phase_request()` (Pre/Executor/Post prompt + transcript).
2. Sends the request via `PhaseTransport::send_streaming()` (streaming) or `send()` (non-streaming).
3. Extracts the normalized model output (visible text, tool calls, usage, stop reason).
4. Checks for tool calls → pauses and stores continuation.
5. Advances the state machine via `OrchestrationState::advance()`.

The runtime is transport-agnostic: it depends only on the `PhaseTransport` trait, not on HTTP or axum. This allows the same runtime to be used with any transport implementation.

4.3 Loopback Transport

The `LoopbackPhaseTransport` in `pigs-proxy` sends phase subrequests via HTTP to the local proxy. It parses both non-streaming JSON and SSE responses from the upstream provider, aggregating:

- Chat: text deltas, tool call deltas (merged by index), finish reason, usage, provider extensions.
- Anthropic: content blocks (text, thinking, signature, `tool_use` with `input_json_delta` accumulation), stop reason, usage.
- Responses: output items (message, reasoning, `function_call`), status, incomplete details, usage.

Malformed streams (missing termination events, unknown content block indices, open blocks at stream end) are reported as typed errors, not silently accepted.

4.4 Body Cleaning

The proxy's `clean_empty_messages` function removes messages with empty content, but preserves assistant messages that contain `tool_calls` (even if their content is empty). For messages with empty content and tool calls, the content is replaced with a single space to satisfy upstream providers that reject empty strings.

4.5 CLI Integration

The CLI connects to the local API server via HTTP using `HttpAgentClient`, which implements the `ApiClient` trait. The CLI sends requests with the real (non-`--pig`) model name, so the proxy routes them through passthrough. The CLI's local `PhasedRuntime` performs Pre→Executor→Post orchestration, with tools executed in-process. This design gives the CLI real SSE streaming while maintaining local tool execution.

5 Related Work

5.1 Multi-Stage LLM Reasoning

ReAct (?) pioneered interleaved reasoning and acting in LLMs. Unlike Pigs, ReAct embeds control flow inside the model's output—the model decides when to reason and when to act. Pigs externalizes control flow to a deterministic state machine, using the model only for semantic content and routing signals.

Reflexion (?) adds verbal reinforcement learning through reflective feedback. The reflection trigger is model-driven, whereas Pigs' Post phase is a mandatory structural step enforced by the state machine.

Self-Refine (?) uses a single LLM as generator, critic, and refiner. Pigs separates these roles into distinct phases with distinct prompts, and adds a Pre (planning) phase that Self-Refine lacks.

Plan-and-Solve (?) separates planning from execution in prompt structure. Pigs extends this to a three-phase model with an explicit review phase and failure-recovery routing.

Tree of Thoughts (?) and **Graph of Thoughts** (?) generalize chain-of-thought to search structures. These approaches require the model to self-evaluate states, while Pigs uses a dedicated Post phase for evaluation.

5.2 LLM-as-Code and Deterministic Control

Recent work has proposed shifting control flow from the LLM to deterministic program structures. **LLM-as-Code** (?) treats the LLM as an adaptive component that cannot alter the program’s execution path. **Compile, Then Page** (?) compiles standard operating procedures into executable pseudo-code with a stack machine. **XFlow** (?) defines a DSL with lifecycle-governed symbols that stage uncertainty. These systems share Pigs’ philosophy of deterministic control flow, but operate at a higher abstraction level (workflow DSLs, compiled programs). Pigs achieves deterministic control at the API middleware level, requiring no DSL or compilation step.

Rel(AI)Build (?) proposes a deterministic control plane for coding agents with a phase state machine. This is architecturally similar to Pigs, but focuses on supply-chain integrity (SHA-256 addressing, HMAC lockfiles) rather than API-level orchestration.

5.3 LLM Middleware and Proxies

Towards a Middleware for LLMs (?) is a vision paper proposing middleware for enterprise LLM deployment, but provides no implementation. Pigs is an implemented system that realizes this vision with a specific orchestration model.

TokenMizer (?) is a transparent proxy with session state management via a typed knowledge graph. While it adds state to the proxy layer, it does not impose phase structure or control markers.

STREAM (?) implements multi-tier LLM inference routing (local→HPC→cloud) with a complexity judge. Pigs’ routing is phase-based (Pre→Executor→Post), not tier-based.

Dyserve (?) is a workflow-aware serving layer that reroutes on tool-call failures. Pigs’ PIGFAIL marker provides a similar failure-recovery mechanism, but at the phase level rather than the tool-call level.

5.4 Tool-Use Agent Frameworks

ToolLLM/ToolBench (?) provides a framework and benchmark for tool-augmented LLMs. **MetaGPT** (?) encodes SOPs into prompt sequences. **MemGPT** (?) manages control flow via OS-inspired interrupts. Pigs differs by keeping tool execution external to the orchestration layer—the runtime pauses and returns tool calls to the client, rather than executing them internally.

5.5 Agent Benchmarks

Several benchmarks evaluate LLM agents: **AgentBench** (?) (8 environments), **SWE-bench** (?) (software engineering), **GAIA** (?) (general AI assistants), **ToolBench** (?) (tool use), and **AgentBoard** (?) (multi-turn analysis). These benchmarks evaluate end-to-end agent performance but do not isolate the contribution of orchestration structure. Pigs’ phased design enables ablation studies (e.g., Pre-only vs. full pipeline) that these benchmarks can measure.

5.6 Surveys and Foundations

Several surveys provide comprehensive background for this work. ? and ? survey LLM-based autonomous agents, covering single-agent and multi-agent scenarios. ? provides a broad survey of large language models. ? surveys tool learning with foundation models, defining a four-stage taxonomy (task planning, tool selection, tool calling, response generation) that maps to Pigs’ Pre→Executor→Post phases. ? proposes cognitive architectures for language agents (CoALA), formalizing the distinction between reasoning, acting, and memory. ? surveys prompt engineering techniques systematically. ? discusses challenges and applications of LLMs.

5.7 Self-Correction and Verification

The Post phase in Pigs is related to LLM self-correction. [?] shows that LLMs cannot reliably self-correct reasoning without external feedback, motivating Pigs’ use of a dedicated review phase with structured prompts rather than relying on the model’s intrinsic self-correction. [?] proposes Self-RAG for retrieval-augmented self-reflection. [?] demonstrates that step-by-step verification improves reasoning. [?] introduces Constitutional AI for harmlessness through AI feedback. [?] improves LLMs with external knowledge and automated feedback.

5.8 LLM Serving Systems

On the infrastructure side, **vLLM** [?] introduces PagedAttention for efficient memory management in LLM serving. **Orca** [?] proposes distributed serving with iteration-level scheduling. **S-LoRA** [?] serves thousands of concurrent LoRA adapters. These systems optimize *inference* performance, while Pigs optimizes *orchestration* structure—the two concerns are orthogonal and complementary.

5.9 Security Considerations

Prompt injection attacks [??] are a concern for LLM middleware. Pigs’ architecture mitigates some risks by separating the orchestration layer from the model’s output: control markers are detected by the middleware, not by the model’s own reasoning, making it harder for injected content to hijack the control flow. However, a comprehensive security analysis is left for future work.

5.10 Classical Foundations

The concept of using a state machine for control flow has deep roots in software engineering. [?] introduced statecharts as a visual formalism for complex systems. Pigs’ Pre→Executor→Post state machine with marker-based transitions can be seen as a lightweight instance of this classical pattern, applied at the LLM API middleware level.

6 Discussion

6.1 Why Text Markers Instead of Tool-Based Handoff?

A natural alternative to text control markers is to inject an orchestration-specific tool (e.g., `finish_task`, `replan_task`) into the model’s tool set. We reject this approach for three reasons. First, it pollutes the tool namespace—the model must distinguish between user-defined tools (`bash`, `read_file`) and orchestration tools, creating confusion. Second, tool calls consume a round-trip: the model emits a tool call, the system executes it (trivially), and the model receives the result. Text markers are zero-overhead. Third, tool-based handoff couples the orchestration logic to the tool-calling protocol, making it protocol-specific. Text markers work universally across all protocols and even with models that do not support tool calling.

6.2 Why Post Stays in Post (Not Back to Executor)?

When the Post phase produces output without a marker (indicating progress but not completion), the state machine continues in Post rather than returning to Executor. This design prevents infinite Executor↔Post loops: if the model oscillates between “do” and “review” without converging, the Post iteration budget will eventually exhaust and return an error. Returning to Executor would reset the loop without progress, creating a potential infinite cycle.

6.3 Why Phase Instructions Only in User Payload?

Phase-specific instructions (Pre questions, Executor plan, Post review criteria) are appended to the current user message, not injected as system prompts. This preserves the client’s original system prompt verbatim, which is important for two reasons. First, many providers cache system prompts for reduced latency and cost [?]; modifying the system prompt would invalidate the cache. Second, the system prompt often contains project-specific instructions (coding standards, safety rules) that should not be overridden by orchestration logic.

6.4 Why HTTP Loopback Instead of In-Process Dispatch?

Phase subrequests could be dispatched in-process (bypassing HTTP), as was done in earlier versions of Pigs. We switched to HTTP loopback for two reasons. First, it ensures that phase subrequests transparently inherit all proxy-level features (channel selection, model mapping, body cleaning, thinking-effort injection, retry). With in-process dispatch, these features must be manually invoked, leading to code duplication and divergence. Second, HTTP loopback makes the control flow explicit and debuggable—each phase subrequest appears in proxy logs as a normal HTTP request, and the internal token distinguishes it from client requests.

6.5 Limitations

Marker robustness. The control-marker approach depends on the model’s ability to follow instructions about when to output `PIGEND`/`PIGFAIL`. In practice, models sometimes output markers in inappropriate contexts (e.g., quoting the marker in prose). The marker detection algorithm mitigates this by checking only the last non-empty line and requiring a preceding reason, but it is not foolproof.

Single-model assumption. The current design uses the same model for all three phases. Multi-model orchestration (e.g., a cheaper model for Pre, a stronger model for Executor) is architecturally possible but not yet implemented.

No formal verification. The state machine is tested with unit tests covering all transitions, but lacks formal verification (e.g., model checking) of properties like “the system always terminates” or “no phase is skipped.”

Protocol coverage. While three major protocols are supported, the system does not cover all provider-specific extensions (e.g., Google Gemini’s API).

7 Conclusion

We presented Pigs, a phased orchestration middleware for LLM API requests that combines a deterministic Pre→Executor→Post state machine with protocol-native request preservation, HTTP loopback transport, and bounded in-memory continuation. By elevating orchestration from a prompting technique to a middleware concern, Pigs separates deterministic control flow from stochastic LLM calls, making the orchestration transparent, debuggable, and protocol-agnostic. The system is implemented as a 13-crate Rust workspace and supports three major API protocols with real SSE streaming.

Future work includes multi-model phase orchestration, formal verification of the state machine, and empirical evaluation on public agent benchmarks (SWE-bench, GAIA, AgentBench) to quantify the impact of phased orchestration on task completion quality.